

Forth Interpreter User Manual

Introduction

This is a simple Forth-like stack-based interpreter implemented in standard C. It provides an educational environment for learning about stack-based computation, dictionary-based execution, and threaded interpretation.

What is Forth?

Forth is a stack-based programming language and environment known for its simplicity, extensibility, and efficiency. This interpreter implements a subset of Forth features suitable for learning and experimentation.

Features

- **Stack-based architecture:** All operations work with a data stack
- **Dictionary system:** Built-in and user-defined words stored in a dictionary
- **REPL interface:** Interactive Read-Eval-Print Loop
- **User-defined words:** Create custom functions using `:` and `;`
- **Control flow:** if-then-else, loops (begin-until, begin-while-repeat, do-loop)
- **Memory operations:** Store and fetch values from memory
- **String operations:** Display strings, convert formats, build counted strings
- **Arithmetic and logical operations:** Complete set of mathematical and comparison operations

Getting Started

Prerequisites

- GCC compiler installed
- Standard C library (included with GCC)

Building the Interpreter

```
gcc forth.c -o forth
```

Running the Interpreter

Interactive mode:

```
./forth
```

Batch mode with a script file:

```
./forth < test.forth
```

Exiting the Interpreter

Type `quit` in the REPL to exit.

Basic Concepts

The Stack

Forth uses a Last-In-First-Out (LIFO) data stack for all operations. Numbers and intermediate results are pushed onto and popped from the stack.

Stack notation: In Forth documentation, stack effects are shown as `( before -- after )` where:

- Items before `--` are consumed from the stack
- Items after `--` are produced on the stack

Example: `+ ( a b -- sum )` means `+` consumes two numbers and produces their sum.

The Dictionary

The dictionary stores all words (functions) known to the interpreter:

- **Built-in words:** Predefined operations like `+`, `dup`, `.`
- **User-defined words:** Custom functions created with `:` and `;`

REPL (Read-Eval-Print Loop)

The interpreter runs in an interactive loop:

1. Reads input from the user

2. Evaluates each word/token
3. Prints results or errors
4. Repeats

Execution Modes

- **Interpret mode** (default): Words are executed immediately
- **Compile mode**: Words are compiled into new definitions (entered with `:`)

Built-in Words

Arithmetic Operations

Word	Stack Effect	Description
<code>+</code>	<code>(a b -- sum)</code>	Add two numbers
<code>-</code>	<code>(a b -- difference)</code>	Subtract b from a
<code>*</code>	<code>(a b -- product)</code>	Multiply two numbers
<code>/</code>	<code>(a b -- quotient)</code>	Divide a by b (integer division)
<code>mod</code>	<code>(a b -- remainder)</code>	Modulo operation
<code>/mod</code>	<code>(a b -- rem quot)</code>	Division with both remainder and quotient
<code>abs</code>	<code>(n -- n)</code>	Absolute value
<code>negate</code>	<code>(n -- -n)</code>	Negate (two's complement)
<code>min</code>	<code>(a b -- min)</code>	Return minimum of two values
<code>max</code>	<code>(a b -- max)</code>	Return maximum of two values
<code>1+</code>	<code>(n -- n+1)</code>	Increment by 1
<code>1-</code>	<code>(n -- n-1)</code>	Decrement by 1
<code>2*</code>	<code>(n -- n*2)</code>	Multiply by 2
<code>2/</code>	<code>(n -- n/2)</code>	Divide by 2

Stack Manipulation

Word	Stack Effect	Description
dup	(a -- a a)	Duplicate top stack item
?dup	(n -- 0 n n)	Duplicate if non-zero
drop	(a --)	Remove top stack item
swap	(a b -- b a)	Swap top two stack items
over	(a b -- a b a)	Copy second item to top
rot	(a b c -- b c a)	Rotate top three items
nip	(a b -- b)	Remove second item
tuck	(a b -- b a b)	Insert copy of top under second
2dup	(a b -- a b a b)	Duplicate top two items
2drop	(a b --)	Remove top two items
2swap	(a b c d -- c d a b)	Swap top two pairs
depth	(-- n)	Return number of items on stack

Comparison Operations

Word	Stack Effect	Description
=	(a b -- flag)	True if a equals b
<	(a b -- flag)	True if a < b
>	(a b -- flag)	True if a > b
<=	(a b -- flag)	True if a <= b
>=	(a b -- flag)	True if a >= b
<>	(a b -- flag)	True if a != b
0=	(n -- flag)	True if n equals zero
0<	(n -- flag)	True if n is negative
0>	(n -- flag)	True if n is positive

Note: Comparison operations return -1 for true, 0 for false.

Constants

Word	Stack Effect	Description
true	(-- -1)	Push boolean true value
false	(-- 0)	Push boolean false value

Logical Operations

Word	Stack Effect	Description
and	(a b -- result)	Bitwise AND
or	(a b -- result)	Bitwise OR
not	(a -- result)	Bitwise NOT (one's complement)
xor	(a b -- result)	Bitwise exclusive OR
lshift	(n u -- result)	Left shift n by u bits
rshift	(n u -- result)	Right shift n by u bits

Memory Operations

Word	Stack Effect	Description
!	(value addr --)	Store value at address
@	(addr -- value)	Fetch value from address
+	(n addr --)	Add n to value at address
c!	(c addr --)	Store byte at address
c@	(addr -- c)	Fetch byte from address
,	(n --)	Compile n into memory, increment HERE
here	(-- addr)	Return next available memory address

I/O Operations

Word	Stack Effect	Description
.	(n --)	Print number (respects current base)
.s	(--)	Print entire stack
cr	(--)	Print newline
."	(--)	Print string literal (usage: ." text")
emit	(c --)	Output character
space	(--)	Output single space
spaces	(n --)	Output n spaces
u.	(u --)	Print unsigned number
.r	(n width --)	Print number right-justified in field
.h	(n --)	Print number in hexadecimal

String Operations

Word	Stack Effect	Description
S"	(-- addr len)	Parse string literal, return address and length
TYPE	(addr len --)	Display string from address+length pair
COUNT	(addr -- addr+1 len)	Convert counted string to modern format
C,	(c --)	Compile byte into memory, increment HERE

Return Stack Operations

The return stack is used for temporary value storage and loop control. **Use with caution** - improper use can break loops and word definitions.

Word	Stack Effect	Description
>r	(n --) (R: -- n)	Move top of data stack to return stack
r>	(-- n) (R: n --)	Move top of return stack to data stack
r@	(-- n) (R: n -- n)	Copy top of return stack to data stack

Warning: Return stack operations must be balanced within word definitions. Items pushed with `>r` must be popped with `r>` before word completion.

Number Base Control

These words change how numbers are input and displayed:

Word	Effect	Description
<code>decimal</code>	Sets base to 10	Default mode for decimal I/O
<code>hex</code>	Sets base to 16	Hexadecimal mode for I/O
<code>binary</code>	Sets base to 2	Binary mode for I/O

Note: Changing the base affects both input parsing and output formatting with `.` word.

Example:

```
decimal 255 .      \ Outputs: 255
hex 255 .          \ Outputs: ff
binary 5 .         \ Outputs: 101
decimal            \ Return to decimal mode
```

Control Flow

Conditional Execution

```
if ... then
if ... else ... then
```

Example:

```
5 3 > if ." Greater" then
10 5 < if ." Less" else ." Greater or equal" then
```

Loops

Begin-Until Loop:

```
begin ... condition until
```

Executes the body until the condition is true.

Example:

```
0 begin dup . 1 + dup 10 = until drop
```

Begin-While-Repeat Loop:

```
begin condition while ... repeat
```

Executes while condition is true.

Example:

```
10 begin dup 0 > while dup . 1 - repeat drop
```

Do-Loop:

```
limit start do ... loop
```

Executes from start to limit-1.

Example:

```
10 0 do i . loop
```

Defining Words

Word	Description
:	Start word definition
;	End word definition
VARIABLE	Create a variable
CONSTANT	Create a constant
CREATE	Create an expandable word

User-Defined Words

Create custom words using colon definitions:

```
: word-name ... ;
```

Example:

```
: square dup * ;  
5 square . \ Prints 25
```

Words can call other words and use control flow:

```
: factorial  
  dup 1 > if  
    dup 1 - factorial *  
  else  
    drop 1  
  then  
;
```

Advanced Features

Variables

Create variables that store values in memory:

```
VARIABLE counter  
10 counter !      \ Store 10 in counter  
counter @ .       \ Print value of counter
```

Constants

Create named constants:

```
42 CONSTANT answer  
answer .          \ Prints 42
```

Return Stack Usage

Use the return stack for temporary storage in complex calculations:

```
: average 2dup + 2 / >r 2drop r> ;  
10 20 average .
```

Prints: 15

```
: process >r dup * r> + ;  
5 10 process .
```

Prints: 35 (5*5 + 10)

Important: Always balance `>r` with `r>` within a word definition!

Number Base Operations

Work with hexadecimal, binary, or decimal:

```
decimal 255 .  
hex 255 .  
binary 5 .  
decimal
```

Outputs: 255, ff, 101

Byte Operations

Store and retrieve individual bytes:

```
VARIABLE name  
65 name c!  
name c@ emit
```

Outputs: A

Memory Increment

Efficiently increment variables:

```
VARIABLE counter
0 counter !
1 counter +!
counter @ .
5 counter +!
counter @ .
```

Outputs: 1, 6

Formatted Output

Create aligned output:

```
: print-row i 5 .r i dup * 5 .r cr ;
5 0 do print-row loop
```

Outputs:

```
0      0
1      1
2      4
3      9
4     16
```

Memory Management

Word	Stack Effect	Description
<code>cells</code>	<code>(n -- bytes)</code>	Convert cells to bytes
<code>allot</code>	<code>(n --)</code>	Allocate n cells of memory

String Handling

Strings in this Forth implementation use ANS Forth conventions:

- **S"** parses string literals and returns address/length pairs
- **TYPE** displays strings from address+length pairs
- **COUNT** converts counted strings (addr+length byte) to modern format
- **C**, builds counted strings byte-by-byte

String Literals:

```
s" Hello, World!" TYPE
```

Counted Strings:

```
CREATE greeting 13 C, 72 C, 101 C, 108 C, 108 C, 111 C, 44 C, 32 C, 87 C, 111  
C, 114 C, 108 C, 100 C, 33 C,  
greeting COUNT TYPE
```

Loop Indices

In do-loops, access the current index:

Word	Stack Effect	Description
i	(-- index)	Get current loop index
j	(-- index)	Get outer loop index

Examples

Basic Arithmetic

```
5 3 + .      \ Prints 8  
10 3 - .     \ Prints 7  
4 5 * .      \ Prints 20  
15 4 / .     \ Prints 3  
15 4 mod .   \ Prints 3
```

Stack Manipulation

```
5 dup . .    \ Prints 5 5  
1 2 3 .s     \ Prints < 1 2 3 >  
1 2 swap .s  \ Prints < 2 1 >  
1 2 over .s  \ Prints < 1 2 1 >
```

User-Defined Functions

```
: add dup + ;      \ Double a number
5 add .           \ Prints 10

: greet ." Hello, World!" cr ;
greet             \ Prints Hello, World!
```

Control Flow Examples

```
\ Conditional execution
10 5 > if ." Greater" then

\ Begin-while-repeat loop
0 begin dup 5 < while dup . 1 + repeat drop

\ Do-loop (prints 0 1 2 3 4)
5 0 do i . loop
```

Variables and Constants

```
VARIABLE x
42 x !
x @ .

100 CONSTANT max-value
max-value .
```

String Operations

```
\ Display a string literal
S" Hello, World!" TYPE

\ Create and display a counted string
CREATE greeting 13 C, 72 C, 101 C, 108 C, 108 C, 111 C, 44 C, 32 C, 87 C, 111
C, 114 C, 108 C, 100 C, 33 C,
greeting COUNT TYPE

\ String in a word definition
: hello S" Hello, Forth!" TYPE ;
hello
```

Error Handling

The interpreter provides non-fatal error handling. Common errors include:

- **Stack underflow/overflow:** Trying to pop from empty stack or push to full stack
- **Division by zero:** Attempting to divide by zero
- **Unknown word:** Referencing undefined words
- **Memory access:** Invalid memory addresses
- **Compilation errors:** Syntax errors in word definitions

When an error occurs:

1. An error message is printed
2. The interpreter state is reset
3. Execution continues

Testing and Debugging

Interactive Testing

Use `.s` frequently to inspect the stack:

```
5 3 + .s    \ Check stack after operations
```

Batch Testing

Create test files and run them:

```
./forth < test.forth
```

Debugging Tips

1. Use `.s` to monitor stack state
2. Test operations incrementally
3. Check for stack underflow errors
4. Verify control flow nesting
5. Use simple test cases first

Example Test File

```
\ Test basic arithmetic
5 3 + . cr
10 2 - . cr

\ Test stack operations
1 2 3 .s cr
dup .s cr
drop .s cr

\ Test user-defined word
: test 2 * 3 + ;
5 test . cr
```

Note: Comments (lines starting with `\`) are shown for clarity but are not currently supported by the interpreter. All text is treated as executable code.

Technical Details

Stack Size

- Data stack: 1024 cells
- Return stack: 1024 cells
- Dictionary: 256 words maximum

Memory Layout

- Memory[0-99]: Variables and user data
- Memory[100-299]: Transient string buffer (200 bytes) for interpret mode
- Memory[300+]: Permanent string storage and compiled data

Data Types

- All values are 64-bit integers (`long long`)
- Memory addresses are also 64-bit
- Strings use address+length pairs or counted format

Performance

- Dictionary lookup: O(n) linear search
- Stack operations: O(1)
- Memory access: O(1)

Limitations

- Fixed memory sizes
- No floating-point arithmetic
- Simple dictionary implementation
- Line-based input processing

Conclusion

This Forth interpreter provides a solid foundation for learning stack-based programming concepts. Start with simple arithmetic and stack operations, then progress to user-defined words and control flow constructs. Experiment freely and use the REPL to explore the language interactively.